

Universidade Federal de Pernambuco - Centro de Informática
Sistemas de Informação

Mateus Ribeiro de Albuquerque

Relatório: Preparação de Dados
O Problema Nomao

Recife
2025

Objetivo

O Objetivo dessa fase foi seguir um algoritmo específico:

1. Seleção dos Dados
2. Limpeza dos Dados
3. Construção dos Dados
4. Integração dos Dados
5. Formatação dos Dados
6. Testes entre fases para validação das mudanças com *Random Forest (Classifier)*

Modificações Iniciais

O primeiro passo foi receber de duas fontes diferentes os dados de Treino e Teste (Separadamente e conforme o *Train Test Split* da fase 2).

```
#Recebendo o dataset
data_path =
os.path.join('C:\\Users\\mateu\\Arquivos
de Programas
Faculdade\\Repositorios\\DataMining-CRISP
-DM-The-Nomao-Problem\\data\\processed',
"nomaoTrain_2.csv")

data = pd.read_csv(data_path)
data2 =
pd.read_csv("C:\\Users\\mateu\\Arquivos
de Programas
Faculdade\\Repositorios\\DataMining-CRISP
-DM-The-Nomao-Problem\\data\\processed\\n
omaoTest_2.csv")

#Criando o DataFrame
df = pd.DataFrame(data)
dfTest = pd.DataFrame(data2)
```

Isso é importante para atualizar simultaneamente ambos os datasets visando um futuro teste de treinamento.

Para ajudar com a visualização tabular dos datasets nessa fase, foi atribuído

para cada atributo "V1", "V2", "V3" ... "VN" seu respectivo nome atribuído no OpenML

(https://www.openml.org/search?type=data&sort=version&status=any&order=asc&exact_name=nomao&id=1486).

Novos index:

```
Index(['clean_name_intersect_min',
'clean_name_intersect_max',
'clean_name_levenshtein_sim',
'clean_name_trigram_sim',
'clean_name_levenshtein_term',
'clean_name_trigram_term',
'clean_name_including',
'clean_name_equality',
'city_intersect_min',
'city_intersect_max',
'city_levenshtein_sim',
'city_trigram_sim'],
dtype='object')
Index(['coordinates_long_trigram',
'coordinates_long_equality',
'coordinates_lat_diff',
'coordinates_lat_levenshtein',
'coordinates_lat_trigram',
'coordinates_lat_equality',
'geocode_coordinates_diff',
'coordinates_diff', 'Class'],
dtype='object')
```

Primeiras e Últimas linhas

	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V110	V111	V112	V113	V114	V115	V116	V117	V118	Class	
0	1.0	1.00	1.000000	1.000000	1.000000	b'2'	b'2'	0.860946	0.821519	...	0.750000	0.500000	b'2'	0.999953	0.777778	0.461538	b'2'	1.0	1.000000	Duplicado	
1	1.0	0.75	0.857143	0.857143	0.854737	0.947368	b'2'	b'1'	0.860946	0.821519	...	0.500000	0.387692	b'2'	0.999983	0.538462	0.437500	b'2'	1.0	1.000000	Duplicado
2	1.0	1.00	1.000000	1.000000	1.000000	b'2'	b'2'	0.860946	0.821519	...	0.666667	0.461538	b'2'	0.999993	0.800000	0.666667	b'2'	1.0	1.000000	Duplicado	
3	1.0	0.75	0.857143	0.857143	0.842185	0.833333	b'2'	b'1'	1.000000	1.000000	...	0.500000	0.285714	b'2'	0.998005	0.555556	0.384615	b'2'	1.0	0.999994	Duplicado
4	0.0	0.00	0.250000	0.000000	0.250000	0.000000	b'1'	b'1'	0.860946	0.821519	...	0.007000	0.396108	b'1'	0.902576	0.644330	0.430854	b'1'	1.0	0.979322	Duplicado

Seleção dos Dados

O primeiro ponto foi a criação de uma função capaz de listar todos os atributos que possuísem uma correlação alta o suficiente para ser considerada redundante, determinada acima de 0.85.

```
#Verificação de Variaveis com
alta correlação

#Selecionar apenas as colunas
numéricas
df_numeric =
df.select_dtypes(include=['float
64', 'int64']).copy()

#Calcular correlação
corr_matrix =
df_numeric.corr().abs()

#Obter pares altamente
correlacionados
threshold = 0.85
corr_pairs = (

corr_matrix.where(np.triu(np.ones
s(corr_matrix.shape),
k=1).astype(bool))
    .stack()
    .reset_index()
    .rename(columns={'level_0':
'Variável 1', 'level_1':
'Variável 2', 0: 'Correlação'})
)

#Filtrar apenas as correlações
fortes
high_corr =
corr_pairs[corr_pairs['Correlaçã
o'] >
threshold].sort_values(by='Corre
lação', ascending=False)
```

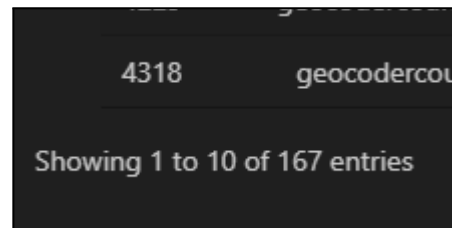
```
pd.set_option('display.max_rows'
, None) # Mostrar todas as
linhas

print("Pares de variáveis
altamente correlacionadas (>
0.85):")
print(high_corr)

itables.show(high_corr)

pd.reset_option('display.max_row
s') #Resetar a opção para o
padrão
```

O resultado demonstrou cerca de um total de **167** Pares de Variáveis que possuíam uma correlação acima de 0.85, uma quantidade alarmante.



Para uma melhor análise, o dataset foi separado em “Famílias”. No nomao, cada instância possui uma gama de atributos pertencentes a um mesmo tipo de ideia. Um exemplo é a família de atributos “Clean_Name” demonstrada a seguir:

```
cols = [
    "clean_name_intersect_min",
    "clean_name_intersect_max",

    "clean_name_levenshtein_sim",
    "clean_name_trigram_sim",

    "clean_name_levenshtein_term",
    "clean_name_trigram_term"
]
```

Basicamente, atributos numéricos que expressam respectivamente:

1. Se o nome por escrito do respectivo local tem uma interseção mínima ou máxima com algum outro local (sobreposição literal)
2. O valor de 0 a 1 da métrica de similaridade gramatical de levenshtein entre o nome do local e algum outro do dataset
3. A exata mesma lógica do anterior porém com a métrica de trigram
4. uma lógica semelhante mas aplicadas a “termos” nas palavras e não a suas strings completas.

Com isso foi realizada algumas visualizações de correlações **específicas entre famílias**:

	clean_name_intersect_min	clean_name_intersect_max	clean_name levenshtein_sim
clean_name_intersect_min	1	0.87216	0.868639
clean_name_intersect_max	0.87216	1	0.969952
clean_name levenshtein_sim	0.868639	0.969952	1
clean_name_trigram_sim	0.921643	0.980805	0.985226
clean_name levenshtein_term	0.711383	0.874895	0.8834
clean_name_trigram_term	0.856403	0.968416	0.964689

clean_name levenshtein_sim	clean_name_trigram_sim	clean_name levenshtein_term	clean_name_trigram_term
0.868639	0.921643	0.711383	0.856403
0.969952	0.980805	0.874895	0.968416
1	0.985226	0.8834	0.964689
0.985226	1	0.865754	0.970739
0.8834	0.865754	1	0.908578
0.964689	0.970739	0.908578	1

Exemplo com Clean name

Percebe-se que a alta correlação entre atributos de mesma família, esse padrão segue para as outras.

Com isso algumas informações foram retiradas, foram identificados diversos grupos de atributos altamente redundantes. Observou-se, por exemplo, que métricas como *geocodercountrynamecode_intersect_min*, *intersect_max*, *levenshtein_sim/term* e *trigram_(sim/term)* apresentaram **correlação perfeita** (1,000) entre si, indicando que todas essas variáveis **medem essencialmente o mesmo comportamento**. Além disso, blocos relacionados a *geocoder* — como *geocoderlocality*, *geocoderinputaddress*, *geocoderoutputaddress* e *geocoderpostalcodenumber_* — aumentam consideravelmente a dimensionalidade do conjunto de dados, **sem acrescentar**

informação nova, já que mantêm correlações **muito altas (entre 0,95 e 0,99)**. O mesmo padrão foi encontrado em atributos de *text matching*, como *clean_name*, *city*, *zip*, *street*, *website* e *countryname*, nos quais ***levenshtein*, *trigram* e *intersect* apresentaram correlações acima de 0,90**, o que é esperado, pois **essas métricas são variações de uma mesma forma de mensurar similaridade textual**.

Outro ponto importante foi a forte relação entre *coordinates_long_* e *coordinates_lat_* (e suas versões *geocode_*), que exibiram correlações de 0,90 a 0,98 — o que faz sentido, já que **quando a longitude é semelhante entre dois registros, a latitude tende a ser também**. De forma análoga, as variáveis *phone_diff*, *phone_levenshtein*, *phone_trigram* e seus equivalentes de *fax_* mostraram-se altamente correlacionadas (0,90–0,97), representando outro bloco redundante.

Diante desses achados, decidiu-se adotar alguns critérios para a seleção final das variáveis. Primeiramente, **priorizaram-se as métricas *trigram***, por serem mais robustas em cenários com ruído textual e variações em nomes e endereços, enquanto as métricas *levenshtein* são mais específicas para pequenas edições de caracteres. Em segundo lugar, **manteve-se as variáveis *including*, por capturarem melhor relações de inclusão entre registros**, sendo úteis para identificar duplicidades reais. Por fim, optou-se pela **remoção de todas as métricas do tipo *_intersect_min/max*, *_levenshtein_term* e *_trigram_term***, por apresentarem alta colinearidade com as métricas principais (*_levenshtein_sim* e *_trigram_sim*) e não contribuírem com informação adicional relevante. Essa redução visa eliminar redundâncias, diminuir a dimensionalidade e evitar problemas de sobreajuste, **mantendo apenas as variáveis *trigram* e *including* como representativas de cada grupo de atributos**.

Sendo assim, **funções para filtragem dos datasets** onde foi removido de todo o dataset os atributos: **"_intersect_min"**,

"_intersect_max", "_levenshtein_term" e "_trigram_term". E um agrupamento de famílias foi realizado.

```
# Agrupando por famílias de atributos, separando coordinates_long
familias = {}
for col in df_filtrado.columns:
    if "_" in col:
        familia = col.split("_")[0]
        familias.setdefault(familia, []).append(col)

for familia, colunas in familias.items():
    print(f"Familia: {familia}, Colunas: {colunas}")

print(f"Total de famílias: {len(familias)}")
```

0.0s

Familia: clean, Colunas: ['clean_name_levenshtein_sim', 'clean_name_levenshtein_term', 'clean_name_trigram_sim', 'clean_name_trigram_term']
 Familia: city, Colunas: ['city_levenshtein_sim', 'city_levenshtein_term', 'city_trigram_sim', 'city_trigram_term']
 Familia: zip, Colunas: ['zip_levenshtein_sim', 'zip_levenshtein_term', 'zip_trigram_sim', 'zip_trigram_term']
 Familia: street, Colunas: ['street_levenshtein_sim', 'street_levenshtein_term', 'street_trigram_sim', 'street_trigram_term']
 Familia: website, Colunas: ['website_levenshtein_sim', 'website_levenshtein_term', 'website_trigram_sim', 'website_trigram_term']
 Familia: countryname, Colunas: ['countryname_levenshtein_sim', 'countryname_levenshtein_term', 'countryname_trigram_sim', 'countryname_trigram_term']
 Familia: geocoderlocalityname, Colunas: ['geocoderlocalityname_levenshtein_sim', 'geocoderlocalityname_levenshtein_term', 'geocoderlocalityname_trigram_sim', 'geocoderlocalityname_trigram_term']
 Familia: geocoderinputaddress, Colunas: ['geocoderinputaddress_levenshtein_sim', 'geocoderinputaddress_levenshtein_term', 'geocoderinputaddress_trigram_sim', 'geocoderinputaddress_trigram_term']
 Familia: geocoderoutputaddress, Colunas: ['geocoderoutputaddress_levenshtein_sim', 'geocoderoutputaddress_levenshtein_term', 'geocoderoutputaddress_trigram_sim', 'geocoderoutputaddress_trigram_term']
 Familia: geocoderpostalcodenumber, Colunas: ['geocoderpostalcodenumber_levenshtein_sim', 'geocoderpostalcodenumber_levenshtein_term', 'geocoderpostalcodenumber_trigram_sim', 'geocoderpostalcodenumber_trigram_term']
 Familia: geocodercountrynamecode, Colunas: ['geocodercountrynamecode_levenshtein_sim', 'geocodercountrynamecode_levenshtein_term', 'geocodercountrynamecode_trigram_sim', 'geocodercountrynamecode_trigram_term']
 Familia: phone, Colunas: ['phone_diff', 'phone_levenshtein', 'phone_trigram']
 Familia: fax, Colunas: ['fax_diff', 'fax_levenshtein', 'fax_trigram']
 Familia: geocode, Colunas: ['geocode_coordinates_long_diff', 'geocode_coordinates_long_levenshtein', 'geocode_coordinates_long_trigram']
 Familia: coordinates, Colunas: ['coordinates_long_diff', 'coordinates_long_levenshtein', 'coordinates_long_trigram']
 Familia: Class, Colunas: ['Class_num']
 Total de famílias: 16

Limpeza dos Dados

Uma longa função de seleção foi formulada da seguinte maneira: para cada coluna por família selecionada a coluna de "Trigram" é enviada para um array "Colunas_selecionadas", caso a coluna trigram não existisse, levenshtein seria selecionada porém isso não foi preciso. Em sequência de maneira análoga, as colunas including de cada família também são enviadas para as colunas selecionadas e por fim, para aqueles atributos que não utilizam nenhuma dessas métricas, procura-se nas famílias a métrica de "equality" ou "diff" que são colineares e se aplicam para atributos como "phone_number".

Para o salvamento, ambos os datasets de treino e Teste (Sempre separados desde o início) são exportados como csv.

Desse modo com uma combinação de métodos baseados em filtragem (Conhecimento prévio manual) e estatística (Correlações > 0.85) o seguinte processo se

seguir:

- ✓ 120 variáveis originais,
- ✗ 44 removidas por regra (_intersect, _term)
- 🔍 16 famílias representadas (trigram/levenshtein)
- ⚠️ limpeza final
- 📦 30 features finais mantidas

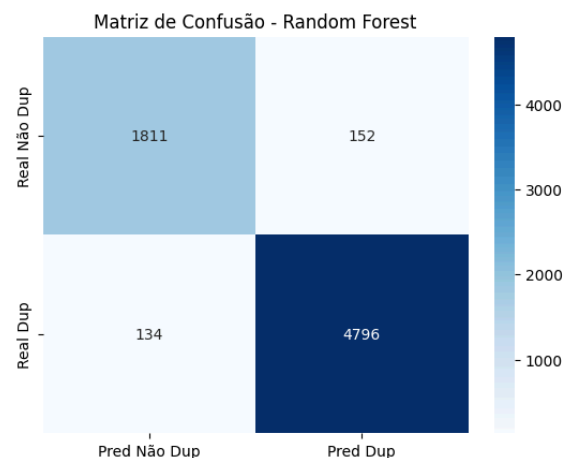
Para finalizar a etapa de "Limpeza" foi feita uma verificação residual de pares que tem correlação maior que 0.85 no novo dataframe e foi visto que **nenhum par apresentou colinearidade maior que 0.85**.

Assim, o primeiro treino de teste, utilizando Random Forest, foi feito para verificar os resultados **antes da construção de dados novos**

Relatório de Classificação:

		precision	recall	f1-score	support
Não Duplicado	Duplicado	0.93	0.92	0.93	1963
	Não Duplicado	0.97	0.97	0.97	4930
accuracy				0.96	6893
macro avg		0.95	0.95	0.95	6893
weighted avg		0.96	0.96	0.96	6893

ROC-AUC Score: 0.9893



Validação cruzada (ROC-AUC): 0.9898 ± 0.0015

Bons números.

Construção de Dados

Buscando ideias para adicionar ao dataset, foi definido a criação de uma métrica que define a média de similaridade textual entre todos os atributos de Trigram e uma Flag que data 1 caso ambos os endereços estejam inclusos.

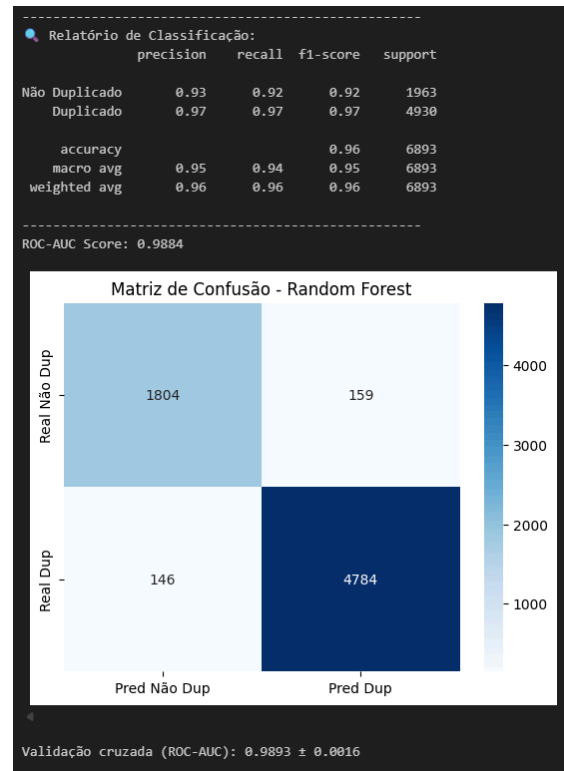
```
#Métrica de Similaridade de
Texto

df_selecionado['avg_text_similar
ity'] =
df_selecionado[['clean_name_trig
ram_sim', 'city_trigram_sim',
'street_trigram_sim',
'zip_trigram_sim']].mean(axis=1)
df_test_selecionado['avg_text_sim
ilarity'] =
df_test_selecionado[['clean_name_
trigram_sim',
'city_trigram_sim',
'street_trigram_sim',
'zip_trigram_sim']].mean(axis=1)

#Métrica de Endereços

df_selecionado['both_address_inc
luded'] =
((df_selecionado['street_includi
ng'] == 2) &
(df_selecionado['zip_including']
== 2)).astype(int)
df_test_selecionado['both_address
_included'] =
((df_test_selecionado['street_inc
luding'] == 2) &
(df_test_selecionado['zip_includi
ng'] == 2)).astype(int)
```

Após a implementação dessas métricas nos datasets, uma nova implementação de um treinamento foi feita para verificar se houve alguma melhora nos resultados:



Percebe-se então que, dentre as poucas mudanças ocorridas:

1. o F1-Score diminuiu em 0.01 em não duplicados
2. O valor ROC-AUC diminuiu em 0.0009
3. E 0,0005 de Diminuição na validação cruzada

Números que parecem insignificantes mas percebe-se que a construção de novos dados fez o modelo cometer **19 erros a mais no total, no modelo de random forest** o que é relativamente significativo. o que pode indicar que para o Nomao a construção de dados novos é prejudicial ao modelo.

Formatação dos Dados

Para realizar a formatação dos dados foi aplicado um **Encoder Ordinal** para os dados categóricos pois, os dados categóricos do Nomao funcionam como uma escala de 1 (Não incluso) a 3 (Inclusão total) com 2 sendo inclusão parcial o que vai de acordo com a lógica do ordinal encoder.

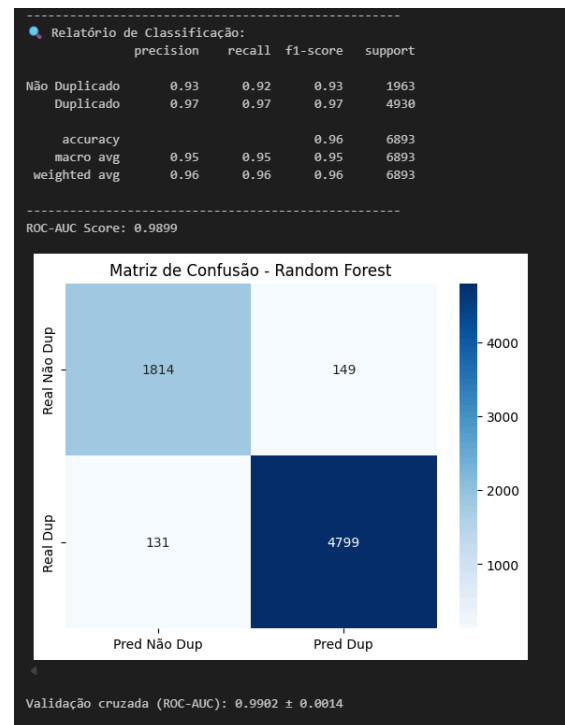
Passando dessa etapa, foi aplicado o **StandardScaler** aos dados.

A escolha desse método se justifica porque, embora as *features* já estivessem em uma faixa próxima de 0 a 1, havia **pequenas variações de amplitude e dispersão** entre variáveis contínuas, como as métricas *trigram_sim* e *diff*.

Essas diferenças poderiam influenciar o peso relativo de cada atributo no processo de aprendizado, especialmente em modelos lineares. Além disso, o **StandardScaler** preserva *outliers* leves, diferentemente do *MinMaxScaler*, que comprime todos os valores para o intervalo [0,1] e pode distorcer variáveis já normalizadas. Outro ponto é que o *StandardScaler* funciona muito bem para **dados quase normalizados**, como é o caso deste conjunto, em que muitas métricas de similaridade apresentam distribuição aproximadamente gaussiana entre 0.6 e 1.0 — nesse cenário, ele apenas centraliza os valores em torno de zero, sem alterar a forma original da distribuição.

Por fim, essa escolha espera facilitar a **comparação entre as variáveis**, já que a padronização pelo *z-score* (média 0 e desvio padrão 1) torna os coeficientes e importâncias das *features* mais interpretáveis e comparáveis entre si, beneficiando futuras análises com modelos diferentes.

Resultado do Treinamento Pós-Formatação dos Dados:



Comparação com dados não formatados:

Métrica	Dados Formatados	Dados Não Formatados
Accuracy	0.96	0.96
F1 (Não Duplicado)	0.93	0.92
F1 (Duplicado)	0.97	0.97
ROC-AUC	0.9899	0.9884
Validação Cruzada (ROC-AUC)	0.9902 ± 0.0014	0.9893 ± 0.0016

Leve melhora nos dados formatados

As métricas de ROC-AUC e validação cruzada foram ligeiramente superiores (diferenças de 0.0008–0.001). Isso mostra que o modelo generaliza melhor, ou seja, se comporta de forma mais consistente entre folds e no conjunto de teste.

O F1-score para a classe “Não Duplicado” subiu de 0.92 → 0.93, com recall também ligeiramente maior (0.92 → 0.93). Isso indica que, após o escalonamento e codificação, o modelo passou a distinguir melhor as instâncias minoritárias (não duplicadas), que são mais difíceis de identificar corretamente. O desvio padrão da validação cruzada reduziu de 0.0016 → 0.0014.

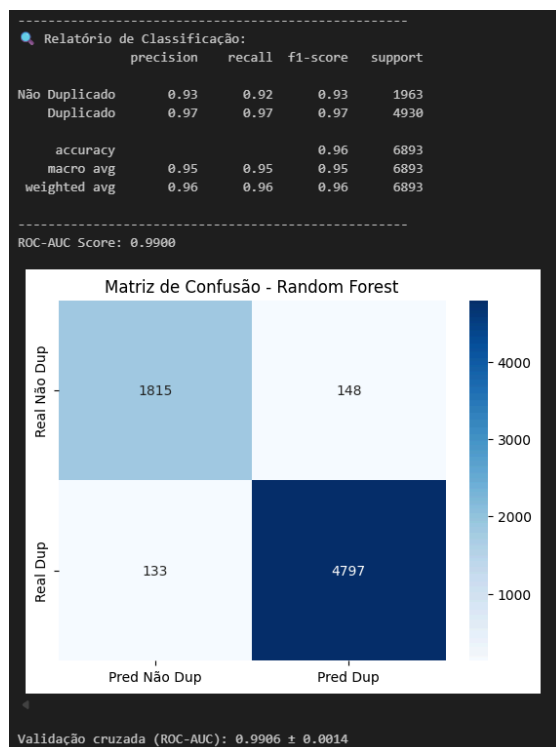
Isso significa que o modelo com dados padronizados é mais estável em diferentes

subconjuntos de treino, o que é um sinal de melhor preparação de dados.

Último Treino de Teste

Como último passo, visto que a única diminuição de performance do modelo está atrelada à inclusão de dados novos, foi realizado um teste final com os datasets de treino e teste **sem** as métricas novas.

Resultados:



Observação final

Foi tentado posteriormente realizar uma técnica de balanceamento de classes (SMOTE e Undersampling), porém ambas as técnicas diminuíram (Minimamente) os valores do modelo então não foi levado adiante.

O modelo sem as colunas novas é ligeiramente melhor em ROC-AUC e CV (diferenças de ~0.0001–0.0004). A versão com colunas novas reduz um pouco os falsos negativos (131 vs 133), mas aumenta os falsos positivos (149 vs 148). As diferenças são minúsculas e dentro do desvio da validação cruzada (± 0.0014) — estatisticamente, não dá para afirmar ganho real ao adicionar as features. Sendo assim, o conjunto mais simples é sempre preferível.